



**UNIVERSITY
OF LONDON**

Module Name: Computer Security (Midterm Coursework)

Module Code: CM2025

Name: Devon Maya Xavior

SID: 200618744

Date: 04 July 2024

PART B: CRYPTOGRAPHY

1(A)

main.py	Save	Run	Output
<pre>1 def encrypt(plaintext, key): 2 plaintext = plaintext.upper() 3 key = key.upper() 4 key_adjusted = key[:len(plaintext)] # key length 5 ciphertext = [] 6 7 for i in range(len(plaintext)): 8 p = ord(plaintext[i]) - ord('A') # Convert letter to number (0-25) 9 k = ord(key_adjusted[i]) - ord('A') # Convert letter to number (0-25) 10 c = (p + k) % 26 # Perform mod addition 11 ciphertext.append(chr(c + ord('A'))) # Convert number back to letter 12 13 return ''.join(ciphertext) 14 15 #Tested with my name 16 plaintext = "DEVONMAYAXAVIOR" # Replaced with my name - DEVON MAYA XAVIOR 17 key = "THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM" 18 ciphertext = encrypt(plaintext, key) 19 print(f"Ciphertext: {ciphertext}") 20</pre>			<pre>Ciphertext: WLDGVEALEUAHXZV === Code Execution Successful ===</pre>

Source Code (in python):

```
def encrypt(plaintext, key):
    plaintext = plaintext.upper()
    key = key.upper()
    key_adjusted = key[:len(plaintext)] # key length
    ciphertext = []

    for i in range(len(plaintext)):
        p = ord(plaintext[i]) - ord('A') # Convert letter to number (0-25)
        k = ord(key_adjusted[i]) - ord('A') # Convert letter to number (0-25)
        c = (p + k) % 26 # Perform mod addition
        ciphertext.append(chr(c + ord('A'))) # Convert number back to letter

    return ''.join(ciphertext)

#Tested with my name
plaintext = "DEVONMAYAXAVIOR" # Replaced with my name - DEVON MAYA XAVIOR
key = "THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM"
ciphertext = encrypt(plaintext, key)
print(f"Ciphertext: {ciphertext}")
```

Input: DEVON MAYA XAVIOR

Output: WLDGVEALEUAHXZV

1(B)

To decrypt the encrypted message using the encryption algorithm in part (A), we have to first take the two inputs – ciphertext and key in uppercase, adjust the key to match the length of the ciphertext, modular SUBTRACTION for each letter in the ciphertext and key, mod 26, the result will be the decrypted message.

main.py	Output
<pre>1 def encrypt(plaintext, key): 2 plaintext = plaintext.upper() 3 key = key.upper() 4 key_adjusted = key[:len(plaintext)] # key length 5 ciphertext = [] 6 7 for i in range(len(plaintext)): 8 p = ord(plaintext[i]) - ord('A') # Convert letter to number (0-25) 9 k = ord(key_adjusted[i]) - ord('A') # Convert letter to number (0-25) 10 c = (p + k) % 26 # Perform mod addition 11 ciphertext.append(chr(c + ord('A'))) # Convert number back to letter 12 13 return ''.join(ciphertext) 14 15 #Tested with my name 16 plaintext = "DEVONMAYAXAVIOR" # Replaced with my name - DEVON MAYA XAVIOR 17 key = "THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM" 18 ciphertext = encrypt(plaintext, key) 19 print(f"Ciphertext: {ciphertext}") 20 21 def decrypt(ciphertext, key): 22 ciphertext = ciphertext.upper() 23 key = key.upper() 24 key_adjusted = key[:len(ciphertext)] # Adjust key length 25 plaintext = [] 26 27 for i in range(len(ciphertext)): 28 c = ord(ciphertext[i]) - ord('A') # Convert letter to number (0-25) 29 k = ord(key_adjusted[i]) - ord('A') # Convert letter to number (0-25) 30 p = (c - k) % 26 # Perform modular subtraction 31 plaintext.append(chr(p + ord('A'))) # Convert number back to letter 32 33 return ''.join(plaintext) 34 35 # Test decryption with the given ciphertext 36 decrypted_plaintext = decrypt(ciphertext, key) 37 print(f"Decrypted Plaintext: {decrypted_plaintext}") 38</pre>	<pre>Ciphertext: WLDGVEALEUAHXZV Decrypted Plaintext: DEVONMAYAXAVIOR === Code Execution Successful ===</pre>

Source code (in python):

```
def decrypt(ciphertext, key):
    ciphertext = ciphertext.upper()
    key = key.upper()
    key_adjusted = key[:len(ciphertext)] # Adjust key length
    plaintext = []

    for i in range(len(ciphertext)):
        c = ord(ciphertext[i]) - ord('A') # Convert letter to number (0-25)
        k = ord(key_adjusted[i]) - ord('A') # Convert letter to number (0-25)
        p = (c - k) % 26 # Perform modular subtraction
        plaintext.append(chr(p + ord('A'))) # Convert number back to letter

    return ''.join(plaintext)

# Test decryption with the given ciphertext
decrypted_plaintext = decrypt(ciphertext, key)
print(f"Decrypted Plaintext: {decrypted_plaintext}")
```

Input: WLDGVEALEUAHXZV

Output: DEVON MAYA XAVIOR

1(C)

There are 2 ways a cipher can be broken where the attacker does not have the key:

1. **Known Plaintext Attack:**

Encrypted email communications – if an attacker intercepts encrypted emails between 2 people, and if the attacker is aware of the structure of emails in general, they can analyse the patterns in the ciphertext and match these patterns with the plaintext structures. It will not be too hard to understand which part is the header or which part is the body and common phrases. The attacker can then use these patterns to analyse the encryption key and hence, decrypt the emails.

2. **Brute-force/Cryptanalysis:**

Certainly! Attackers can sometimes decrypt encrypted data without possessing the encryption key by exploiting weaknesses in the encryption algorithm itself. This can involve systematic testing of multiple keys, exploiting patterns in the encrypted data, or using advanced statistical methods. These vulnerabilities highlight the importance of using robust encryption algorithms and secure implementation practices to ensure data security.

1(D)

Three key factors for defining a robust key to enhance algorithm security:

1. The key must be generated using secure random number generation techniques so that keys are truly random and not predictable, in other words, we need high entropy in the key.
2. Key must be long enough to resist brute-force attacks. For symmetric encryption, key lengths of at least 128 bits are recommended.
3. The key needs to have high cryptographic strength as well, which means that it should include differential and linear cryptanalysis, related-key attacks etc.

2(a)

The purpose of padding in cryptography algorithms such as DES is to primarily make sure that the plaintext or message matches the requirements of the encryption algorithm, especially when the length of the plaintext is not a multiple of the block size used by the algorithm.

An example is when we have a DES encryption algorithm that works on 64-bit blocks. If the plaintext that has to be encrypted is not 64 bits, we will need to pad it to make it reach the required block size. The purpose of DES padding is also to add information to ensure integrity.

2(b)

Step 1: Replace “YYY” with first 3 letters of my name will give “DEVComputerSecurity”.

Step 2: Replace plaintext to hexadecimal will give

44 45 56 43 6F 6D 70 75 74 65 72 53 65 63 75 72 69 74 79

Step 3: calculating the padding needed

Since the length of the plaintext here is 19 (152 bits), another 5 bytes of padding (40 bits) is needed to reach 8 bytes (64 bits)

Step 4: add padding – adding 5 bytes (40 bits) of zeros will give

44 45 56 43 6F 6D 70 75 74 65 72 53 65 63 75 72 69 74 79 00 00 00 00 00

Step 5: results

44 45 56 43 6F 6D 70 75 74 65 72 53 65 63 75 72 69 74 79 00 00 00 00 00

2(c)

To encrypt the plaintext M=“ComputerSecurity”, we have to first convert the plaintext into a number suitable for RSA encryption, using ASCII.

Step 1: convert each letter in the plaintext into ASCII representation:

C: 67, o: 111, m: 109, p: 112, u: 117, t: 116, e: 101, r: 114, S: 83, e: 101, c: 99, u: 117, r: 114, i: 105, t: 116, y: 121

Step 2: Concatenate it into a single number and add padding

Original Plaintext (ASCII numeric form):

67 111 109 112 117 116 101 114 83 101 99 117 114 105 116 121

After RSA-PKCS#1 v1.5 Padding:

0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00

67 111 109 112 117 116 101 114 83 101 99 117 114 105 116 121

2(d)

A digital signature is like a virtual stamp that confirms the sender of a message is legitimate and make sure that the message sent has not been altered.

Yes, we can use RSA to generate digital signatures. The sender can use their personal secret key to make a unique stamp or signature for the intended message. The receiver will then use the sender's

public key to confirm the stamp or signature's validity. If both match, it shows that the sender is valid and that the message has not been changed.

As for DES, no we cannot use it for digital signatures, this is because DES uses a shared key for both encryption and decryption, which makes it hard to be used as a signing validator. Also, DES is mainly used for encrypting data, not really to validate who the sender is.